Rural Healthcare AI

Codebase Analysis & Technical Deep Dive

Generated on: 2026-02-01 20:57

1. System Overview

This system is an offline-capable, multi-agent AI pipeline designed for rural healthcare. It ingests multimodal data (audio, images, text, docs), stabilizes it into a canonical format, maintains a longitudinal patient history in a vector database (Qdrant), and safely retrieves similar historical cases to aid decision-making.

2. Solution Pipeline (6-Agent Workflow)

The system operates as a directed acyclic graph (DAG) managed by LangGraph. Data flows sequentially through six specialized agents:

Agent	Role	Key Output
1. Field Ingestion	Normalizes raw inputs, transcribes audio, assesses quality.	Canonical `patient_data` JSON
2. Patient Memory	Generates embeddings, updates Qdrant, summarizes history.	Updated `patient_current_state`
3. Context Builder	Plans retrieval by defining hard constraints & filters.	Structured `retrieval_plan`
4. Case Retrieval	Executes hybrid search (Dense + Sparse) with re-ranking.	List of `retrieved_cases`
5. Explanation	Synthesizes cases into a human-readable explanation (LLM).	Safety-checked `explanation_text`
6. Referral	deterministic logic to detect care gaps & recurring risks.	Actionable `followup_nudges`

3. Deep Dive: Component Logic

Detailed analysis of the core files in `my_code/`.

File: agents.py

Implements the Field Ingestion Agent. Focuses on data stabilization and quality assessment.

Component	Implementation Details
Class Name	FieldIngestionAgent
Processors	• TextProcessor: Normalizes whitespace, flags sparse text. • AudioProcessor: Uses OpenAI Whisper for transcription. • ImageProcessor: Uses Laplacian variance for blur detection. • DocumentProcessor: Tesseract OCR & PDF extraction.
Uncertainty Flags	Calculates boolean flags: text_sparse, audio_noisy, image_unclear, history_partial.
Output	Produces a sanitized PatientData object/dict ready for storage.

File: patient_memory_agent.py

Manages long-term memory. Handles embedding generation and Qdrant interactions.

Component	Implementation Details
Class Name	PatientMemoryAgent

Embeddings	Uses MultiModalEmbedder to vectorise text (Sentence-BERT), images (CLIP), and audio (CLAP/Optional).
Storage Strategy	Upserts event payload to Qdrant collection patient_context_events_v1. Payload includes timestamp, program, demographics, and uncertainty flags.
State Generation	Retrieves **all** history for the patient to generate `patient_current_state`, summarizing total visits, recurring themes (via keyword extraction), and aggregate uncertainty.

File: context_builder_agent.py

The 'Retrieval Planner'. Prevents unsafe search by defining strict constraints before search happens.

Component	Implementation Details
Class Name	ContextBuilderAgent
Signal Extraction	Extracts deterministic signals from history: Age Group (Pediatric vs Adult), Pregnancy Status (Pregnant/None), Program Type.
Constraint Building	Creates a `retrieval_plan` containing: • **Hard Filters**: Program match, Age range (±10y), Pregnancy match. • **Geography**: Restricts search to 'same_block' by default. • **Modality Policy**: Disables audio/image search if quality flags are raised.
Safety	Does NOT perform the search itself. Acts as a safety gate.

File: similar_case_retrieval_agent.py

Executes the Hybrid Search logic defined by the Context Builder plan.

Component	Implementation Details
Class Name	SimilarCaseRetrievalAgent
Hybrid Search	Combines two search methods: 1. **Dense**: Vector similarity (Cosine/DotProduct) on text embeddings. 2. **Sparse**: BM25-style keyword matching using `recurring_themes`.
Re-ranking Logic	Determinstic formula: Score = (0.6 * Dense) + (0.4 * Sparse) + PayloadBoost. • Boosts score for matching Village (+0.1) or Program (+0.05). • Penalizes high uncertainty.
Filtering	Applies Qdrant Filters (must/must_not) derived from the retrieval plan. **Crucially**, it self-excludes the current event ID.

4. Infrastructure & Utilities

File	Description
api.py	FastAPI backend. Defines endpoints (`/process`, `/search`) and constructs the LangGraph workflow graph. Manages state transitions between agents.
db.py	SQLite database manager. Handles storage of: Raw files (`uploads/`), Transaction logs, JSON dumps, and final outputs. Ensures full audit trail.
qdrant_manager.py	Wrapper for Qdrant Client. Manages connection, collection creation, and search queries.
visu/generate_*.py	Utility scripts for generating PDF reports (like this one) using ReportLab.

End of Report